

Diagram Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 5

1. Diagrams as data . . . 6

1.1. The N by N problem is a metamodel problem . . . 6

1.2. A diagram already carries semantics . . . 6

1.3. Four uses . . . 7

1.4. What diagrams as data make possible . . . 7

2. From picture to model . . . 9

2.1. Three rungs . . . 9

2.2. Meaning without a target model . . . 9

2.3. Depiction is many-to-many . . . 10

2.4. Depiction can be computed . . . 10

2.5. Why the reference has this name . . . 10

3. Prior art . . . 11

4. What is not here, on purpose . . . 13

4.1. What is here . . . 13

II: The How . . . 14

5. Getting started . . . 15

5.1. Adding the model . . . 15

5.2. A resource set . . . 15

5.3. What the build does . . . 16

6. The model . . . 17

6.1. DiagramElement . . . 17

6.2. Diagram and Layer . . . 18

6.3. Node, Port and Label . . . 18

6.4. Connectable and Connection . . . 19

6.5. Point and Bounds . . . 19

7. Building a diagram . . . 20

7.1. The factory . . . 20

7.2. Navigating . . . 22
8. Diagrams as resources . . . **23**
8.1. Saving and loading . . . 23
8.2. Writing diagrams by hand . . . 23
8.3. Fragments . . . 25
9. From Draw.io . . . **27**
9.1. Loading the figure . . . 27
9.2. A converter . . . 27
9.3. The result . . . 29
10. Semantic elements . . . **31**
10.1. Mapping . . . 31
10.2. Checking a diagram against its model . . . 32
10.3. From the depicted object back to the diagram . . . 32
10.4. Many to many . . . 33
III: Looking Ahead . . . **34**
11. What comes next . . . **35**
11.1. Notation models, loaders and generators . . . 35
11.2. Layout . . . 35
11.3. Computed depiction . . . 35
11.4. Diagrams that people and programs both edit . . . 36
11.5. Sites, executable diagrams and assistants . . . 36
Appendix A: Model reference . . . **37**
A.1. Inherited from the runtime's core model . . . 37
Appendix B: Building this book, and editing its figure . . . **39**
B.1. The figure . . . 39
Appendix C: Glossary . . . **40**

Colophon

Diagram Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the diagram model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.diagram:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/diagram`. Every listing in it is included from the module's tests or test resources, which are compiled and run when the book is built. Its figure is a PNG image with a Draw.io diagram embedded, which the tests convert into the diagram model and check against the model it depicts.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Diagram Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Eclipse, EMF, Sirius and GMF are trademarks of the Eclipse Foundation. OMG is a trademark of the Object Management Group. draw.io is a trademark of JGraph Ltd. Visio and PowerPoint are trademarks of Microsoft Corporation. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/diagram/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr

Books.

Preface

This book is about a small model of diagrams. It holds what diagram notations have in common: shapes and where they sit, what connects to what, how elements nest and which layer they are on, what each of them is documented as, and, where there are any, which elements of other models each shape depicts. A diagram element is a model element in its own right, so a diagram is a model even when it has been mapped to nothing.

The first part of the book is the why: why diagrams should be data, how much meaning they can carry, what has been built before, and what this model leaves out on purpose. The second part is the how, for Java developers. The third says what the model is for beyond this edition.

Who this book is for

Architects and analysts who keep diagrams in Draw.io, Visio or PowerPoint and want them to be more than pictures: searchable, reviewable, comparable, and connected to the models of the systems they describe. Part one is for you, and needs no code.

Developers who convert diagrams between notations, generate them from models, check them, or map them to domain models. Read the book in order.

Tool builders who need a neutral, typed structure in the middle of a diagram pipeline.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The runtime has a book of its own, *Nasdanika SDK Runtime: More Structure with Less Effort*, and the runtime's Draw.io module has a Zephyr Book, *Draw.io*. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

The model comes from the diagram model of Nasdanika's model collection, rebuilt on the runtime's core model. This book replaces the model's readme, and keeps its argument.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.diagram`, its Maven coordinates are `org.nasdanika.sdk.runtime.models.diagram:model:2026.11.0`, and its namespace URI is `runtime.sdk.nasdanika.org/models/diagram`.
- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.

Conventions

- **Every listing is included from a test** of the module, or from a resource that a test loads, and is compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **The figure is a diagram.** It is a PNG image with a Draw.io diagram embedded. Open it in Draw.io to edit it; the tests convert it into the diagram model and check it against the model it depicts (Chapter 10).
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.



The readme this book replaces described the model as it was in Nasdanika's model collection: `DiagramElement` extending the requirements model's `Satisfier`, a `tags` feature, the nxcore model, the model tower and its floors, and `semanticElementsSelector`. The runtime's model has none of them yet; `DiagramElement` extends the runtime core model's `NamedElement`, `StringIdentity` and `Attributed`, and the `Satisfier` supertype is a TODO in the Xcore. The readme's `properties` are `attributes`, from core's `Attributed`, since 2026-10-25. This book describes the model as it is, and the rest as planned.

Part I: The Why

Why diagrams should be data, how much meaning a diagram can carry, what others have built, and what this model leaves out on purpose. No code is needed to read this part.

Chapter 1. Diagrams as data

Organizations draw a great deal. Architecture in Draw.io, processes in Visio, strategies in PowerPoint, and sketches on whiteboards photographed with a phone. Most of it ends up as files that nothing reads: a PNG pasted into a wiki page, a `.drawio` file in a repository, a deck nobody can search. The information in those files is real, and expensive to produce, and most of it is lost to everything but the human eye.

This model is a bet that diagrams should be data.

1.1. The N by N problem is a metamodel problem

Every diagramming tool has an export format and a handful of converters to the formats its users complain about most. Draw.io to PlantUML, Visio to Draw.io, PowerPoint to anything: each pair is written separately, and each loses something different. With N notations, that is up to N times N converters, most of which nobody writes.

A shared model in the middle turns them into N loaders and N generators. It also makes the thing in the middle useful in itself. A parse tree that lives for the duration of a conversion helps nobody; a model can be saved, compared, queried, validated and reviewed. When a conversion goes wrong, you look at the model in between and see where.

1.2. A diagram already carries semantics

When someone draws a box labeled "Payments API", the box **is** an architecture element. The rectangle, the fill color and the 240 by 120 pixels are the least interesting facts about it. The file format keeps those facts and throws away the meaning, which survives only in the label and in the heads of the people who drew it.

The model keeps the meaning in two ways, and either is enough.

- **The element itself.** A diagram element has an identity, a name, documentation and attributes, before it is mapped to anything. A diagram is a tree and a graph of named, documented elements, which is structurally close to what an architecture model is.
- **The elements it depicts.** `semanticElements` points from a diagram element to any objects of any other model: an application in an architecture model, an asset in a threat model, a work

item, a class of a metamodel. The picture and the meaning become one object graph, instead of two artifacts and a naming convention between them.

1.3. Four uses

The model's position in the middle gives it four uses:

- **Interchange and conversion.** Load a diagram from one notation and generate another, through a model you can inspect.
- **Diagrams that were never mapped.** A documented, queryable, reviewable model of the diagrams an organization already has, with nothing configured and no domain metamodel at all.
- **Executable diagrams.** Diagrams whose elements carry behavior, run by one interpreter whatever notation they were drawn in.
- **Semantic mapping.** Diagram elements turned into, or linked to, the objects of domain models, by one mapping that works for every notation.

One mapper, one executor, many notations. An executable PowerPoint deck is not a joke in this arrangement: it is the same interpreter with a different loader.

1.4. What diagrams as data make possible

- **Documentation sites** generated straight from diagrams: a page per shape, a link per connection, documentation and ownership attached, and markers pointing back to the source file.
- **Review and diff.** "What changed in this architecture diagram" answered with a list of added nodes and re-pointed connections, instead of two images side by side.
- **Impact queries.** Retiring a service becomes a query over every diagram that depicts it, rather than a hunt through image files.
- **Modernization.** Legacy diagram estates, such as Visio libraries, decades of decks and the canvases of integration platforms, mined for what they document and regenerated on a current stack.
- **Diagrams for and from AI assistants.** A typed, documented structure is a far better contract for a language model than mxGraph XML: an assistant can emit one, and a program can

check it before anything is drawn.

- **Layout as a service.** With topology, bounds, ports and waypoints in one model, a layout engine lays out diagrams from any notation.
- **Mergeable diagrams.** A diagram generated from a source of truth and then improved by people can be regenerated without losing their work, by a three-way merge on the model (Chapter 11).

The rest of part one is about how much structure that takes, and how little.

Chapter 2. From picture to model

Between a picture and a domain model there is a gradient. The model is designed so that a team can stop anywhere on it, and move on later without redrawing.

2.1. Three rungs

Pixel-deep. Boxes, arrows and labels. What most diagrams are, and a perfectly reasonable place to be. Loaded into the model, a pixel-deep diagram is already a tree of elements with names, bounds and connections, which a program can walk.

Diagram as model. The same boxes, now with stable IDs, documentation, properties such as owners and stages, and layers that mean something. You can generate a site from it, search it, compare two versions of it in a pull request, or run it as an executable diagram. Its elements stay untyped: the diagram **is** the model.

Semantically mapped. `semanticElements` points into a domain model, and the diagram becomes a view of an estate that other tools read and write as well. The links may be drawn by hand, produced by a mapping run, or, in a later edition, computed: a shape declares a query instead of a list.

Each rung is optional and incremental, and none invalidates the work done on the one before. Deciding to map a diagram later does not mean redrawing it: the identity and documentation added on the middle rung are exactly what a mapping keys on.

2.2. Meaning without a target model

The narrow reading of `semanticElements` is that a shape is merely a pointer: the shape is notation, the meaning lives elsewhere, look over there. That reading is too narrow, because plenty of diagrams are never mapped to a formal model, and should not have to be.

Give the elements of a diagram stable identity, documentation and a few properties, and you have something worth generating from, searching and reviewing, with no mapping and no domain metamodel at all. Nasdanika's Draw.io site template makes exactly this bet: a `.drawio` file whose elements carry a little documentation and a handful of properties becomes a navigable, searchable documentation site. Nothing was typed, and a great deal became useful.

And the top rung stays available throughout. A shape can depict an architecture element **and** carry documentation of its own. `semanticElements` adds a dimension rather than replacing one.

2.3. Depiction is many-to-many

A shape is not always one thing. A box labeled "Forest" depicts a collection of trees. A container stands for a group of people or a pool of resources. A shape represents an iteration over something.

So `semanticElements` is multi-valued. Single-valued depiction, OMG DI's `modelElement` or GMF's `View.element`, forces such cases either into a fake aggregate object in the domain model or into a naming convention in a property. Here the box points at the members, and the other direction is a query: "which shapes depict this tree" and "what does this box cover" are both answerable (Chapter 10).

2.4. Depiction can be computed

Enumerating a collection by hand does not scale. A "Forest" box with four hundred hand-maintained references is worse than no model at all, because it is wrong within a week.

The answer is a shape that declares **what** it depicts rather than **which objects**: every tree in this package, every service tagged `payments`, the contents of a directory, the results of a SQL or JQL query. The shape stops being a snapshot and becomes a standing question. The explicit list stays primary; a selector would be another source of links, not a replacement for drawing them. In this edition, computed depiction is **PLANNED** (Chapter 11).

2.5. Why the reference has this name

OMG's Diagram Interchange calls the reference `modelElement`, which works only if the diagram is **not** a model. Here it is one: a diagram element is a model element of the runtime's core model, with an ID, documentation and markers. So the reference is named for the distinction that matters, the one that Eclipse Sirius draws between the **semantic** model and the **representation** model.

Chapter 3. Prior art

Diagram interchange is an old problem, and several good attempts have been made at it. This chapter says what the model takes from each, and what it leaves.

The OMG stack: Diagram Definition (DD), Diagram Commons (DC), Diagram Interchange (DI), and the interchange specifications of UML, BPMN and SysML. The nearest prior art and the direct inspiration. DD got the central abstraction right, a notation element that **depicts** a model element, and then wrapped it in a split between abstract and concrete syntax, a style hierarchy, and per-notation DI packages that in practice exist only inside modeling tools. BPMN DI is the success case, and even there interchange between tools is famously partial. Taken here: the depiction reference, renamed and made multi-valued; bounds; waypoints; layers. Left: everything that made DD a specification to implement rather than a model to use.

Native formats of tools: mxGraph XML (Draw.io), VSDX (Visio), OOXML and DrawingML (PowerPoint), Lucidchart, Miro, Figma, Excalidraw. These are the sources, not the rivals. Each is expressive, each is an island, and each has a conversion story that is pairwise and lossy. None has a notion of "the domain element this shape stands for", so the semantics live in the text of shapes and in the reader's head.

Diagrams as code: Mermaid, PlantUML, Graphviz DOT, D2, the Structurizr DSL. Text first, version-controlled, and genuinely good. Their limit is that the source **is** the artifact: there is no object graph to query, no stable identity to reference from a risk register, and cross-references between diagrams are string matching. They make excellent input and output formats for a model, and a poor substitute for one.

Graph interchange: GraphML, GEXF, DOT, Cytoscape JSON, JSON Canvas. They solve the topology half, nodes, edges and attribute bags, and stop there. No layers, no ports, no depiction, no documentation, no provenance, and an untyped property bag where a metamodel should be. Fine for handing a graph to a layout or analysis library; not a place to keep diagrams.

Eclipse modeling: GMF's notation model, Sirius, Graphiti, GLSP. The closest technical relatives. GMF's notation model is the same idea, a `View` with an `element` reference into the semantic model. Two differences. Purpose: those are the persistence formats of diagram editors, bound to one editing tool, while this model is an interchange layer with no editor of its own; the editor is Draw.io, or Visio, or PowerPoint, or whatever the team already uses. Direction: a GMF `View`

without its semantic element is an orphan, because the notation model exists to render something else; here an unmapped element is a first-class documented element, and mapping is an upgrade rather than a precondition.

Canvas and presentation services: Miro, Mural, FigJam, Lucid, Canva, Google Slides. Superb at real-time collaboration, and their export is a picture. They are where diagrams are made, and where diagrams go to stop being data. They too can be sources.

The actual competitor is the same as everywhere else: a PNG pasted into a wiki page, a diagram file in a repository that nothing parses, and a deck whose contents nobody can query.

Chapter 4. What is not here, on purpose

The model is small: ten classes. What it leaves out matters as much as what it has.

No styling. No fill, no stroke, no font, no theme, no z-order, no animation, no slide transition. Those differ from notation to notation and rarely survive a conversion, so they belong to the models of each notation, which extend this one, and to attributes.

No editor. The model is not the persistence format of a diagram editor. People draw in the editors they already have, and loaders bring their diagrams into the model.

No notation. UML, BPMN, C4 and an enterprise's own conventions are all diagrams to this model. What a shape means is either documented on the element, or depicted through `semanticElements`, or both.

Attributes instead of metamodel changes. Every diagram element has `attributes`, a map from strings to strings. They absorb what a source notation carries and the model does not: a Draw.io style, a PowerPoint placeholder index, the URI of a processor, a mapping rule. Supporting a new notation or a new interpreter is then a matter of a loader and some conventions, not a new release of the metamodel.

The core defines the vocabulary that survives conversion, not the pixels that do not.

4.1. What is here

- **Geometry:** `Point`, and `Bounds`, a point with a width and a height.
- **Structure:** `DiagramElement`, the base of everything, with `children` for nesting; `Diagram`, which is a document, a page, a slide or a group; and `Layer`, visible or not, locked or not.
- **Notation:** `Node`, a shape with bounds and ports; `Port`, an attachment point on a node in relative coordinates; `Connection`, from a source to a target through waypoints; and `Label`, a node whose content is text. Nodes and ports are `Connectable`.
- **Meaning:** `semanticElements` on every element, and everything a diagram element inherits from the runtime's core model: an ID, a name, documentation, markers that record where an element came from, an icon and URIs.

Chapter 6 takes them one at a time.

Part II: The How

The model in Java and in files: adding it to a project, its classes, building diagrams, storing them as YAML, converting them from Draw.io, and mapping their elements to the models they depict.

Chapter 5. Getting started

The model is an EMF model, defined in Xcore and generated into Java when the runtime is built. Using it takes a Maven dependency, a line in `module-info.java`, and a resource set.

5.1. Adding the model

The runtime's models have a bill of materials of their own, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, as the runtime's own BOM is imported, and depend on `org.nasdanika.sdk.runtime.models.diagram:model` without a version. For YAML and JSON files, add the runtime's `org.nasdanika.sdk.runtime:emf-json` as well.

The model is a named Java module:

Listing 1. The module declaration of the diagram model

```
import org.eclipse.emf.ecore.EPackage;
import org.nasdanika.sdk.runtime.models.diagram.services.DiagramEPackageProvider;

module org.nasdanika.sdk.runtime.models.diagram {

    exports org.nasdanika.sdk.runtime.models.diagram;
    exports org.nasdanika.sdk.runtime.models.diagram.impl;
    exports org.nasdanika.sdk.runtime.models.diagram.util;

    requires transitive org.nasdanika.sdk.runtime.models.core;

    provides EPackage with DiagramEPackageProvider;

}
```

It requires the runtime's core model transitively, which brings EMF and the runtime's `common` module with it. It provides its `EPackage` as a service, so a resource set configured by the runtime knows the model without registration code.

5.2. A resource set

`NasdanikaResourceSet.createAndConfigure()` creates a resource set and lets every module on the module path contribute to it: models register their packages, and `emf-json` registers resource factories for `.json`, `.yaml` and `.yml` files. A diagram is then saved and loaded like any EMF model (Chapter 8).

You need no resource set to build a diagram in memory, convert it or query it. The factory, `DiagramFactory.eINSTANCE`, creates elements, and the generated interfaces do the rest (Chapter 7).

5.3. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/diagram/diagram.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin, so the model builds from the command line and in any editor; Eclipse is not required. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.diagram`, their implementations in `impl`, and the switch and adapter factory in `util`.

Chapter 6. The model

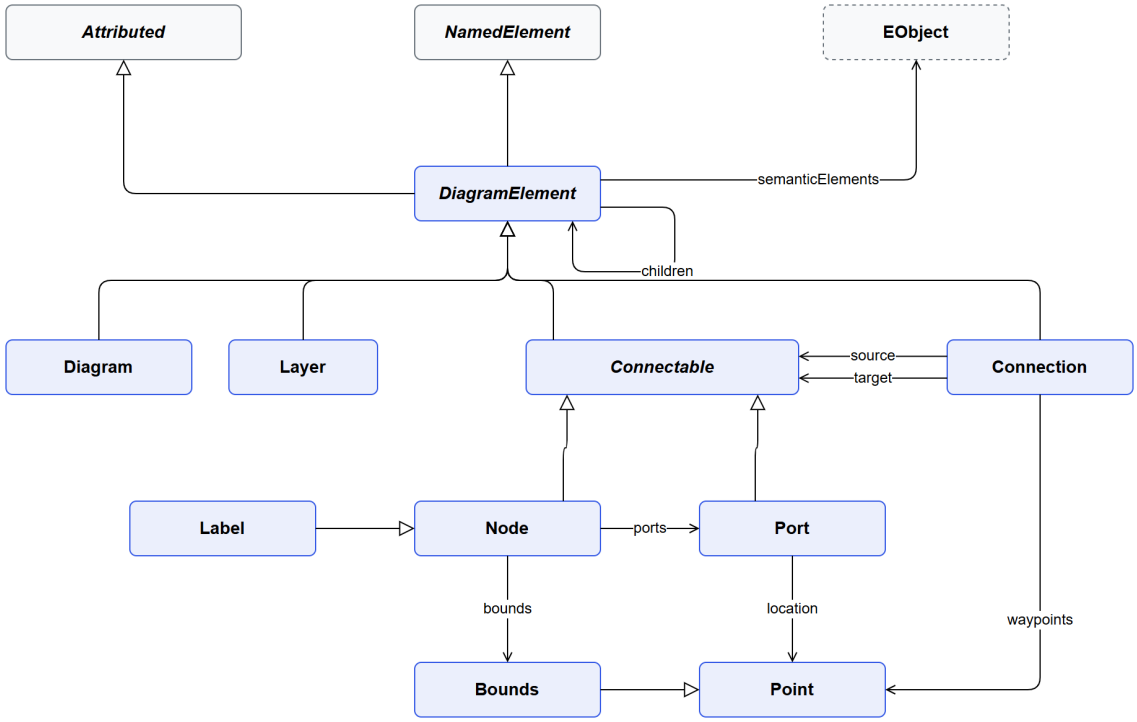


Figure 1. The diagram model. Blue classes are the model's; gray ones come from the runtime's core model and from EMF. Italic names are abstract. Open arrows are references; closed arrows are generalizations.

The figure is drawn in Draw.io, and it is data: Chapter 10 converts it into the diagram model and checks every box and arrow against the model it depicts.

6.1. DiagramElement

The base of everything in a diagram, and abstract. It extends `NamedElement`, `StringIdentity` and `Attributed` of the runtime's core model, so every diagram element has:

- `id` and `name`;
- documentation: `documentation` text with a `docFormat`, a `docRef` to a document elsewhere, or structured `docContents` and `docSections`;
- `markers`, which record where an element came from: a file and a position, or a Git repository, branch and commit;

- an `icon`, and `uris` that identify the element across models and versions;
- `attributes`, a map from strings to strings, for whatever a source notation carries and the model does not, and for conventions of your own (Chapter 4). The same mixin carries the attributes of the runtime's other models, so a convention looks the same on a diagram and on a document.

It adds two features:

`semanticElements`

: The objects of other models that the element depicts. Any `EObject`, as many as it takes (Chapter 10).

`children`

: The elements it contains. Containment is how documents contain pages, pages contain layers, and shapes contain shapes.

6.2. Diagram and Layer

A `Diagram` is a diagram element whose meaning is "a picture, or a set of pictures". Because diagrams nest, the same class is a document of pages, a page, a deck of slides, a slide, or a group of shapes.

A `Layer` is a set of elements shown, hidden and locked together, with `visible`, which defaults to `true`, and `locked`. Layers are the one piece of editor state worth keeping in a neutral model, because they carry meaning: the future-state layer, the out-of-scope layer, the locked background, the layer of conflicts that a merge left for a person to settle.

6.3. Node, Port and Label

A `Node` is a shape: `bounds` for its position and size, and `ports`. Bounds are relative to the node's parent, as in the editors the model is loaded from.

A `Port` is a named attachment point on a node, with a `location` in coordinates relative to the node's size, from 0 to 1 in each dimension: (0.5, 1) is the middle of the bottom edge. A connection attached to a port stays attached to that point of the node after a conversion, instead of snapping to the center.

A `Label` is a node whose content is text, in `text`.

6.4. Connectable and Connection

A `Connection` runs from a `source` to a `target` through `waypoints`, which preserve its route. Either end can be missing, for a connection that is drawn but not attached.

Nodes and ports are `Connectable`: they have `outgoingConnections` and `incomingConnections`, the opposites of `source` and `target`. EMF keeps both sides in step, so setting a connection's source adds the connection to the source's outgoing connections, and topology can be navigated from either end without scanning the diagram.

6.5. Point and Bounds

`Point` has `x` and `y`; `Bounds` is a point with a `width` and a `height`. They are the geometry of the OMG Diagram Commons, and all the geometry the model has.

Chapter 7. Building a diagram

Diagrams usually come from somewhere: an editor, through a loader, or a model, through a generator. Both end up calling the same factory, so building one by hand is the best way to see the model at work.

The running example is the system context of an internet banking system, after the C4 model's well-known example: a customer, the Internet Banking System, and the mainframe it calls, through an API port.

7.1. The factory

Listing 2. Building the system context (`BuildingTests.java`)

```
static Diagram systemContext() {
    DiagramFactory factory = DiagramFactory.eINSTANCE;

    Diagram diagram = factory.createDiagram(); ①
    diagram.setId("internet-banking");
    diagram.setName("Internet Banking System");

    Diagram page = factory.createDiagram();
    page.setId("context");
    page.setName("System Context");
    diagram.getChildren().add(page);

    Node customer = node("customer", "Personal Banking Customer", 240, 0, 200, 80);
    customer.getAttributes().put("type", "person"); ②
    Node ibs = node("ibs", "Internet Banking System", 240, 200, 200, 90);
    ibs.getAttributes().put("type", "software-system");
    ibs.getAttributes().put("owner", "Digital Channels");
    ibs.setDocumentation("Lets customers view their accounts and make payments."); ③
    Node mainframe = node("mainframe", "Mainframe Banking System", 240, 400, 200, 90);
    mainframe.getAttributes().put("type", "software-system");
    mainframe.getAttributes().put("owner", "Core Banking");
    page.getChildren().addAll(List.of(customer, ibs, mainframe));

    Port api = factory.createPort(); ④
    api.setId("api");
    api.setName("API");
    Point bottom = factory.createPoint();
    bottom.setX(0.5);
    bottom.setY(1);
    api.setLocation(bottom);
    ibs.getPorts().add(api);

    page.getChildren().add(connection("uses-ibs", "Uses", customer, ibs));
```

```
    Connection calls = connection("ibs-mainframe", "Gets account information from", api,
            mainframe);
    calls.getAttributes().put("protocol", "XML/HTTPS");
    page.getChildren().add(calls);
    return diagram;
}

static Node node(String id, String name, double x, double y, double width, double height) {
    Node node = DiagramFactory.eINSTANCE.createNode();
    node.setId(id);
    node.setName(name);
    Bounds bounds = DiagramFactory.eINSTANCE.createBounds();
    bounds.setX(x);
    bounds.setY(y);
    bounds.setWidth(width);
    bounds.setHeight(height);
    node.setBounds(bounds);
    return node;
}

static Connection connection(
        String id,
        String name,
        Connectable source,
        Connectable target) {
    Connection connection = DiagramFactory.eINSTANCE.createConnection();
    connection.setId(id);
    connection.setName(name);
    connection.setSource(source);
    connection.setTarget(target);
    return connection;
}
```

① The document is a diagram, and so is its page: diagrams nest.

② Attributes are an EMF map: `getAttributes()` returns an `EMap<String, String>`, with `put`, `get` and `map()` for a `java.util.Map` view.

③ Documentation comes from the runtime's core model.

④ A port on the bottom edge of the Internet Banking System, from which the connection to the mainframe leaves.

Nodes and connections are children of the page here. A page with layers would have the layers as its children and the nodes in the layers; a group would have its members as children.

7.2. Navigating

Connections are maintained from both ends:

Listing 3. Connections, from either end

```
Diagram page = (Diagram) diagram.getChildren().getFirst();
Node ibs = (Node) page.getChildren()
    .stream()
    .filter(e -> "ibs".equals(e.getId()))
    .findFirst()
    .orElseThrow();

assertThat(ibs.getIncomingConnections()) ①
    .extracting(DiagramElement::getId)
    .containsExactly("uses-ibs");
Port api = ibs.getPorts().getFirst();
assertThat(api.getOutgoingConnections().getFirst().getTarget().getId())
    .isEqualTo("mainframe");
assertThat(ibs.getOutgoingConnections()).isEmpty(); ②
```

① Setting the connection's target added it to the node's incoming connections.

② The connection to the mainframe leaves from the port, so it is the port's outgoing connection, not the node's.

For everything else, the generated API is ordinary EMF: `eContents()` and `eAllContents()` walk containment, `eContainer()` goes up, and the reflective API, `eGet` and `eSet`, works on any feature by name. `EcoreUtil` copies, compares and searches.

Chapter 8. Diagrams as resources

A diagram in the model is an EMF object tree, so it is saved and loaded like any other model: as XMI, JSON or YAML, and compared, copied and versioned with EMF's tools.

8.1. Saving and loading

Listing 4. Saving the system context as YAML and loading it back (`BuildingTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(); ①
Path path = Path.of("target", "book", "internet-banking.yaml").toAbsolutePath();
Files.createDirectories(path.getParent());
Resource resource = resourceSet.createResource(URI.createFileURI(path.toString()));
resource.getContents().add(systemContext());
resource.save(null);

Resource loaded = NasdanikaResourceSet.createAndConfigure()
    .getResource(URI.createFileURI(path.toString()), true);
assertThat(EcoreUtil.equals(loaded.getContents(), resource.getContents())).isTrue();
```

① A resource set configured by the runtime: the diagram model registers its package, and `emf-json` the YAML format, because both are on the module path.

`EcoreUtil.equals` compares two object trees feature by feature, references included, which makes it the natural assertion for a round trip, and the basis of a diff.

8.2. Writing diagrams by hand

YAML is readable enough to write diagrams in. This file is the system context of Chapter 7, written by hand:

Listing 5. The system context in YAML (`internet-banking.yaml`)

```
eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Diagram
id: internet-banking
name: Internet Banking System
children:
- eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Diagram
  id: context
  name: System Context
  children:
  - eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Node
    id: customer
    name: Personal Banking Customer
```

```
    attributes:
      type: person
    bounds: { x: 240, "y": 0, width: 200, height: 80 }
  - eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Node
    id: ibs
    name: Internet Banking System
    documentation: Lets customers view their accounts and make payments.
    attributes:
      type: software-system
      owner: Digital Channels
    bounds: { x: 240, "y": 200, width: 200, height: 90 }
    ports:
    - id: api
      name: API
      location: { x: 0.5, "y": 1 }
  - eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Node
    id: mainframe
    name: Mainframe Banking System
    attributes:
      type: software-system
      owner: Core Banking
    bounds: { x: 240, "y": 400, width: 200, height: 90 }
  - eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Connection
    id: uses-ibs
    name: Uses
    source: { $ref: "//@children[id='context']/@children[id='customer']" }
    target: { $ref: "//@children[id='context']/@children[id='ibs']" }
  - eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Connection
    id: ibs-mainframe
    name: Gets account information from
    attributes:
      protocol: XML/HTTPS
    source:
      eClass: https://runtime.sdk.nasdanika.org/models/diagram#//Port
      $ref: "//@children[id='context']/@children[id='ibs']/@ports[id='api']"
    target: { $ref: "//@children[id='context']/@children[id='mainframe']" }
```

Three things to know about the format:

- `eClass` names an object's class by the namespace URI of the model and the class's name. It can be left out where the type of a feature says it, as for `bounds`, `location` and `ports`.
- References are `$ref` with a URI fragment, relative to the file. A fragment can name elements by ID, `@children[id='ibs']`, rather than by position, which keeps references valid when elements are reordered.
- `"y"` is quoted because YAML 1.1 reads a bare `y` as `true`.

The incoming and outgoing connections are not written: EMF sets them from the connections' sources and targets as the file loads. The test loads the file and compares it with the diagram

built in Java:

Listing 6. Loading the hand-written diagram

```
URI uri = URI.createURI(getClass().getResource("internet-banking.yaml").toString());
Resource authored = NasdanikaResourceSet.createAndConfigure().getResource(uri, true);
assertThat(EcoreUtil.equals(authored.getContents().getFirst(), systemContext())).isTrue();
```

8.3. Fragments

Every object in a resource has a URI: the resource's URI and a fragment that is a path from the root.

Listing 7. Finding an element by fragment

```
Node ibs = (Node) loaded.getEObject("//@children[id='context']/@children[id='ibs']"); ①
assertThat(ibs.getName()).isEqualTo("Internet Banking System");
assertThat(EcoreUtil.getURI(ibs).fragment()).isEqualTo("//@children.0/@children.1"); ②
```

① A path of IDs finds the element, whatever its position.

② The fragment EMF generates for the element uses positions.

A fragment by position is valid only as long as nothing is inserted before the element. For references that should survive edits, write fragments by ID, as in the YAML above, or reference elements by their `uris`.



The Xcore declares `children` and `ports` with `keys ^id`, which should make EMF generate fragments by ID, as in the hand-written YAML. The generated `DiagramPackageImpl` registers no keys, so `keys ^id` was dropped without a warning; lookup by ID works only because EMF matches `[id='…']` against the attribute by name. The telemetry model's keys, on attributes declared in its own classes, are generated. The difference is that `id` is declared in the core model's `StringIdentity`, in another package; declaring `StringIdentity` as a direct supertype of `DiagramElement` did not change it. Find out whether Xcore resolves keys inherited from another package, and add the keys in code if it does not, then update this section.

The YAML written by `emf-json` also contains both sides of every connection, `outgoingConnections` and `incomingConnections` as well as `source` and `target`, with

> references by position. Declaring the `Connectable` side `transient` would leave one side in files, the side people write.

Chapter 9. From Draw.io

Most diagrams start in an editor. This chapter loads one from Draw.io: the figure of Chapter 6, which is a PNG image with the diagram embedded.

The runtime's Draw.io module reads and writes Draw.io documents in Java, and its Zephyr Book, *Draw.io*, describes it. Here it is only a source: a document of pages, layers, nodes and connections, with labels, tooltips and properties.

9.1. Loading the figure

Listing 8. Loading the figure and converting it (`SemanticTests.java`)

```
static Diagram figure() throws Exception {
    URL png = SemanticTests.class.getResource("diagram-model.png");
    Document document = Document.loadFromPngMetadata(png).getFirst(); ①
    return new DrawioToDiagram().convert(document);
}
```

① A PNG image can embed several diagrams; the figure embeds one.

9.2. A converter

A loader for a notation maps its structure to the model's. For Draw.io the mapping is close to one to one, and a minimal converter fits on two pages:

Listing 9. A minimal converter from Draw.io to the diagram model (`DrawioToDiagram.java`)

```
public class DrawioToDiagram {

    private final DiagramFactory factory = DiagramFactory.eINSTANCE;

    /** Draw.io elements to the diagram elements made from them, to connect connections */
    private final Map<ModelElement<?>, DiagramElement> converted = new LinkedHashMap<>();

    public Diagram convert(Document document) {
        Diagram diagram = factory.createDiagram();
        for (Page page: document.getPages()) {
            Diagram pageDiagram = factory.createDiagram(); ①
            pageDiagram.setId(page.getId());
            pageDiagram.setName(page.getName());
            diagram.getChildren().add(pageDiagram);
            for (org.nasdanika.sdk.runtime.drawio.Layer<?> drawioLayer:
                    page.getModel().getRoot().getLayers()) {
```

```
                Layer layer = factory.createLayer();
                copy(drawioLayer, layer);
                layer.setVisible(drawioLayer.isVisible());
                pageDiagram.getChildren().add(layer);
                convert(drawioLayer.getElements(), layer);
            }
        }

        converted.forEach((source, target) -> { ②
            if (source instanceof org.nasdanika.sdk.runtime.drawio.Connection drawioConnection
                    && target instanceof Connection connection) {
                connection.setSource(end(drawioConnection.getSource()));
                connection.setTarget(end(drawioConnection.getTarget())); ③
            }
        });
        return diagram;
    }

    private void convert(List<LayerElement<?>> elements, DiagramElement parent) {
        for (LayerElement<?> element: elements) {
            if (element instanceof org.nasdanika.sdk.runtime.drawio.Node drawioNode) {
                Node node = factory.createNode();
                copy(drawioNode, node);
                Geometry geometry = drawioNode.getGeometry();
                Bounds bounds = factory.createBounds();
                bounds.setX(geometry.getX());
                bounds.setY(geometry.getY());
                bounds.setWidth(geometry.getWidth());
                bounds.setHeight(geometry.getHeight());
                node.setBounds(bounds);
                parent.getChildren().add(node);
                convert(drawioNode.getElements(), node); ④
            } else if (element
                    instanceof org.nasdanika.sdk.runtime.drawio.Connection drawioConn) {
                Connection connection = factory.createConnection();
                copy(drawioConn, connection);
                for (var drawioPoint: drawioConn.getPoints()) {
                    Point point = factory.createPoint();
                    point.setX(drawioPoint.getX());
                    point.setY(drawioPoint.getY());
                    connection.getWaypoints().add(point);
                }
                parent.getChildren().add(connection);
            }
        }
    }

    private void copy(ModelElement<?> source, DiagramElement target) {
        target.setId(source.getId());
        String label = source.getLabel();
        if (label != null && !label.isBlank()) {
            target.setName(Jsoup.parse(label).text()); ⑤
        }
        String tooltip = source.getTooltip();
```

```
        if (tooltip != null && !tooltip.isBlank()) {
            target.setDocumentation(tooltip);
        }
        for (String name: source.getPropertyNames()) {
            if (!"id".equals(name)) {
                target.getAttributes().put(name, source.getProperty(name));
            }
        }
        converted.put(source, target);
    }

    private Connectable end(org.nasdanika.sdk.runtime.drawio.Connectable end) {
        return end == null ? null : (Connectable) converted.get(Util.getConnectableNode(end));
    }

}
```

① Every Draw.io page becomes a diagram inside the document's diagram, and every layer a layer.

② Connections are connected in a second pass, when every node has been converted, so a connection can point to a node that comes after it in the file.

③ Draw.io lets connections leave from connection points on a node; `Util.getConnectableNode` returns the node either way.

④ Draw.io nodes contain nodes and connections, and so do the model's.

⑤ Labels in Draw.io are often HTML. The name keeps their text.

What the minimal converter leaves out, a complete one adds: Draw.io's connection points as ports, labels on connections as `Label` children, tags and styles as attributes, links between pages, and markers that record the file and page each element came from. None of it needs a change to the model.

9.3. The result

Listing 10. The converted figure

```
Diagram diagram = figure();
Diagram page = (Diagram) diagram.getChildren().getFirst();
assertThat(page.getName()).isEqualTo("Diagram Model");

List<Node> nodes = all(diagram, Node.class);
assertThat(nodes).hasSize(13);
Node node = nodes.stream().filter(n -> "Node".equals(n.getName())).findFirst().orElseThrow();
assertThat(node.getAttributes().map()).containsEntry("package", "diagram"); ①
assertThat(node.getDocumentation()).isEqualTo("A shape with bounds and ports");
```

```
assertThat(node.getOutgoingConnections()) ②
    .extracting(DiagramElement::getName)
    .contains("ports", "bounds");
```

① The Draw.io properties of the box, here the package of the class it shows, are the element's attributes.

② The figure's arrows are connections of the model, navigable from the node.

The converted diagram is a model with thirteen documented nodes and eighteen connections, and no idea yet what any of them means. The next chapter tells it.



The converter is test code, written for this book. A Draw.io model module, migrated with this model, is meant to provide the loader for real; when it does, replace this chapter's converter with it and keep the listing short. Writing this chapter found a defect in the Draw.io module: `Connection.getPoints().get(index)` threw for every valid index. It is fixed, with a test, in `ConnectionPointListImpl`.

Chapter 10. Semantic elements

The figure of Chapter 6 depicts a metamodel: each box is a class, each open arrow a reference, each closed arrow a generalization. The converted diagram knows none of that. This chapter maps it, then uses the mapping to check the figure against the model it depicts.

10.1. Mapping

A mapping decides, for each diagram element, what it depicts. In the figure, a box's label is the name of a class, and its `package` property says which package to find it in; an arrow's label is the name of a reference of its source's class.

Listing 11. Mapping the figure to the classes and references it depicts (`SemanticTests.java`)

```
static void map(Diagram diagram) {
    Map<String, EPackage> packages = Map.of(
        "diagram", DiagramPackage.eINSTANCE,
        "core", CorePackage.eINSTANCE,
        "ecore", EcorePackage.eINSTANCE);

    for (Node node: all(diagram, Node.class)) {
        EPackage ePackage = packages.get(node.getAttributes().get("package"));
        node.getSemanticElements().add(ePackage.getEClassifier(node.getName())); ①
    }

    for (Connection connection: all(diagram, Connection.class)) {
        if ("reference".equals(connection.getAttributes().get("relationship"))) {
            EClass source = (EClass) connection.getSource().getSemanticElements().getFirst();
            String name = connection.getName();
            EReference reference = (EReference) source.getEStructuralFeature(name);
            connection.getSemanticElements().add(reference); ②
        }
    }
}
```

① The node now depicts an `EClass`: of the diagram model, of the runtime's core model, or of EMF itself.

② A connection depicts an `EReference`. Generalization arrows depict no object of their own; the generalization is a fact about their ends.

A mapping can be this small because the diagram was drawn with it in mind: labels are names, and a property disambiguates. That is the middle rung of Chapter 2 doing its job: the identity and properties added for people are what the mapping keys on.

10.2. Checking a diagram against its model

With every element mapped, the figure can be checked:

Listing 12. Checking the figure against the model it depicts

```
for (Connection connection: all(diagram, Connection.class)) {
    EClass source = (EClass) connection.getSource().getSemanticElements().getFirst();
    EClass target = (EClass) connection.getTarget().getSemanticElements().getFirst();
    switch (connection.getAttributes().get("relationship")) {
    case "extends" -> assertThat(source.getEAllSuperTypes()).contains(target);
    case "reference" -> {
        EReference reference = (EReference) connection.getSemanticElements().getFirst();
        assertThat(reference.getEReferenceType()).isEqualTo(target);
        String opposite = connection.getAttributes().get("opposite");
        if (opposite != null) {
            assertThat(reference.getEOpposite().getName()).isEqualTo(opposite);
        }
    }
    default -> throw new AssertionError("Unknown relationship of " + connection.getId());
    }
}

List<EObject> depicted = all(diagram, DiagramElement.class)
    .stream()
    .flatMap(element -> element.getSemanticElements().stream())
    .toList();
assertThat(depicted).containsAll(DiagramPackage.eINSTANCE.getEClassifiers()); ①
```

① Every class of the diagram model is depicted by some element of the figure.

Each generalization arrow is a supertype, each reference arrow is a reference to the class at its target, with the opposite the arrow claims, and no class of the model is missing from the figure. Rename a class, add one or move a reference, and this test fails until the figure is redrawn. The same pattern checks any diagram that depicts something checkable: an architecture diagram against the inventory of systems, a deployment diagram against the cluster, a process diagram against the workflow engine.

10.3. From the depicted object back to the diagram

`semanticElements` points from the diagram to the model. The other direction is a query:

Listing 13. Which elements depict the class Node?

```
Collection<Setting> settings = EcoreUtil.UsageCrossReferencer.find(
        DiagramPackage.Literals.NODE,
```

```
        diagram); ①
List<String> depictions = settings
    .stream()
    .filter(setting -> setting.getEStructuralFeature()
            == DiagramPackage.Literals.DIAGRAM_ELEMENT__SEMANTIC_ELEMENTS)
    .map(setting -> ((DiagramElement) setting.getEObject()).getName())
    .toList();
assertThat(depictions).containsExactly("Node");
```

① `UsageCrossReferencer` finds every reference to an object within a tree, a resource or a resource set.

For objects of the runtime's core model, `getReferrers(reference)` answers the same question from the object itself, across its resource set. With diagrams loaded into the same resource set as the models they depict, "which diagrams show this service" is one call.

10.4. Many to many

A diagram element can depict several objects, and an object can be depicted by several elements, on several diagrams. A box labeled "Payments" may depict three services and a team; the same service may appear on a context diagram, a deployment diagram and a slide. Both are ordinary in the model: `semanticElements` is a list, and the inbound query returns every element that depicts an object.

Part III: Looking Ahead

What the model is for beyond this edition: notation models, layout, computed depiction, and merging diagrams that people and programs both edit.

Chapter 11. What comes next

The model in this edition is the hub without most of its spokes. This chapter says what is planned around it. Everything in it is **PLANNED** unless it says otherwise.

11.1. Notation models, loaders and generators

Models of each notation extend this one with what makes the notation itself: Draw.io with its styles and magic properties, Presentation with what slide decks share, PowerPoint with layouts and placeholders, Visio with masters and stencils. They are being migrated into the runtime alongside this model. Each brings a loader into the diagram model, and where it makes sense a generator out of it, so that a conversion from any notation to any other passes through one model.

11.2. Layout

A diagram generated from a model, or merged from two, has no good positions. The runtime's `diagram/elk` module lays out the diagram model with the Eclipse Layout Kernel: layered, tree, force-directed and radial algorithms, configured through the elements' attributes, with the results written back into bounds, port locations, waypoints and labels. Because layout works on this model, it applies to every notation that loads into it. It is **AVAILABLE**, and its Zephyr Book, *Diagram Layout with ELK*, generates class diagrams of this model, lays them out and renders them with React Flow and Draw.io.

11.3. Computed depiction

`semanticElementsSelector`, a selector that computes what an element depicts instead of listing it: every class in a package, every service with a tag, the results of a query. The runtime's core model already has evaluators for Groovy, SpEL, XPath and JSR-223 scripts, documented and marked like any model element, so a selector is an evaluator stored on the element. A derived feature would then join the listed and the selected elements.

11.4. Diagrams that people and programs both edit

The failure of every generated diagram is the second run. A diagram is generated from a source of truth, people improve it, by grouping boxes, annotating a connection, moving a cluster where it reads better, and the next generation flattens all of it. So the diagram is generated once and frozen, and living documentation becomes documentation again.

The fix is a three-way merge on the model: the previous generation as the base, the new generation as one side, and the edited diagram as the other. It needs what a model has and a picture does not: stable identity, so that an element is recognized as the same element in all three versions, with IDs, URIs and markers as anchors. What the source still says is refreshed, what people added is kept, and what has disappeared from the source is retired rather than dropped.

Packaged as a command, the merge is also a Git merge driver, registered in `.gitattributes`, which receives the same three versions. Two people editing one diagram on two branches is then the same merge. Conflicts become data: elements on a conflicts layer, which can be hidden, queried across a repository and deleted when settled; or, page by page, four pages, the base, ours, theirs and the merged page with its conflicts, settled in the editor by looking at the diagram rather than at XML. Layout, re-run over the result, keeps the merged diagram readable.

11.5. Sites, executable diagrams and assistants

Documentation sites generated from diagrams in any notation, executable diagrams whose processors are wired along the model's connections, and diagrams given to AI assistants as a typed structure rather than an image, are all planned on top of the model. The runtime's Draw.io Zephyr Book describes the same continuum from the Draw.io side.

Appendix A: Model reference

Namespace URI `runtime.sdk.nasdanika.org/models/diagram`, package `org.nasdanika.sdk.runtime.models.diagram`.

Class	Extends	Features
`DiagramElement`, abstract	`NamedElement`, `StringIdentity` and `Attributed` of the core model	`attributes`, from `Attributed`: map of strings to strings. `semanticElements`: any `EObject`, many. `children`: `DiagramElement`, many, contained.
`Diagram`	`DiagramElement`	None of its own. A document, a page, a slide or a group.
`Layer`	`DiagramElement`	`visible`: boolean, `true` by default. `locked`: boolean.
`Connectable`, abstract	`DiagramElement`	`incomingConnections`: `Connection`, many, opposite of `target`. `outgoingConnections`: `Connection`, many, opposite of `source`.
`Node`	`Connectable`	`bounds`: `Bounds`, contained. `ports`: `Port`, many, contained.
`Port`	`Connectable`	`location`: `Point`, contained, relative to the node, 0 to 1.
`Connection`	`DiagramElement`	`source`, `target`: `Connectable`. `waypoints`: `Point`, many, contained.
`Label`	`Node`	`text`: string.
`Point`		`x`, `y`: double.
`Bounds`	`Point`	`width`, `height`: double.

A.1. Inherited from the runtime's core model

From	Features
`StringIdentity`	`id`, and `getReferrers(reference)`, from `Referrable`, which finds the objects that refer to this one through a reference, across the resource set.
`Documented`	`documentation`, `docFormat`, `docRef`, `docContents`, `docSections`.
`Marked`	`markers`: where the element came from, such as a file and a position or a Git commit.
`ModelElement`	`icon`, `uris`.

From	Features
`NamedElement`	`name`.

Appendix B: Building this book, and editing its figure

This book is AsciiDoc in `models/diagram/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/diagram/model -am verify                          # the model, its tests and this
book
mvn -pl models/diagram/model verify -Dzephyr.editor-notes=false   # without editor's notes
mvn -pl models/diagram/model verify -Dasciidoctor.skip=true       # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/diagram/tests/book` and of the YAML file next to the figure in `src/test/resources`. A listing line holds at most 96 characters, and a warning fails the build.

B.1. The figure

The figure, `diagram-model.png` in `src/test/resources/org/nasdanika/sdk/runtime/models/diagram/tests/book`, is a PNG image with the Draw.io diagram embedded. Open it in Draw.io, edit it, and save it as PNG with *Include a copy of my diagram* checked and a zoom of 200 percent.

The figure follows conventions that its test relies on: a box's label is a class name and its `package` property is `diagram`, `core` or `ecore`; an arrow's label is a reference name, its `relationship` property is `reference` or `extends`, and an `opposite` property names the opposite reference. After changing the model, run the tests: the figure test fails until the figure depicts the model again.

Appendix C: Glossary

Connectable

What a connection can start and end at: a node or a port.

Depiction

The relationship between a diagram element and the objects of other models it stands for, held in `semanticElements`.

Diagram element

Anything in a diagram: a diagram, a layer, a node, a port, a connection or a label. A model element in its own right, with an ID, a name and documentation.

Generator

Code that writes a diagram of some notation from the model.

Hub

The model's position between notations: N loaders and N generators instead of N by N converters.

Loader

Code that reads a diagram of some notation into the model.

Marker

A record of where an element came from: a file and a position, or a Git repository and commit.

Port

A named attachment point on a node, in coordinates relative to the node's size.

Representation model

The model of how something is drawn: this model. The counterpart of the semantic model.

Semantic model

The model of what is drawn: an architecture, a metamodel, a process. Diagram elements point to it through `semanticElements`.

Waypoint

A point a connection is routed through.